

BASIC PHP SYNTAX AND SCRIPT



PHP TAGS

- PHP code is embedded within HTML using special PHP tags. The standard tag is `<?php ... ?>`. Anything written inside these tags is processed by the server before the page is sent to the browser. This allows you to mix dynamic PHP code with static HTML content.

Example:

```
<?php  
    echo "Hello, world!";  
?>
```

- The **echo** statement inside the tags outputs text to the browser. Anything outside the PHP tags is treated as normal HTML and is sent directly to the client without processing.

STATEMENTS AND SEMICOLONS

- In PHP, each instruction the server must execute is called a **statement**, and most statements must end with a semicolon (;).

Example:

```
<?php  
    echo "Hello, world!";  
?>
```

- The semicolon tells the server that this line of code is complete. Forgetting it will cause syntax errors, preventing the script from running.

COMMENTS

- Comments are notes in the code that are ignored by PHP. They are used to explain what a section of code does or to temporarily disable code.
- Single-line comments:

```
// This is a comment  
# This is also a comment
```

- Multi-line comments:

```
/*  
  This is a multi-line comment  
  which spans multiple lines  
*/
```

VARIABLES

- Variables in PHP store data like text, numbers, or other values. Every variable name starts with the **\$** symbol. PHP is loosely typed, which means you don't need to declare the type of data in advance.

```
$name = "Alice";  
$age = 25;
```

- Variables are temporary storage in PHP scripts—they exist only while the script runs unless saved in a database or session.

DATA TYPES

PHP supports several data types to classify information:

- **String** – sequence of characters ("Hello").
- **Integer** – whole numbers (25).
- **Float / Double** – numbers with decimals (3.14).
- **Boolean** – true or false values, often used in condition checks.
- **Array** – a collection of multiple values stored under one variable.

```
$colors = array("Red", "Green", "Blue");
```

BASIC SCRIPT EXAMPLE

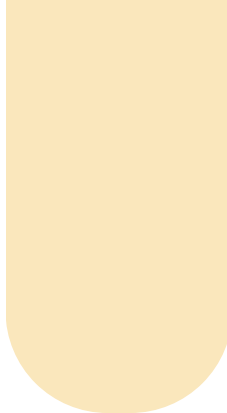
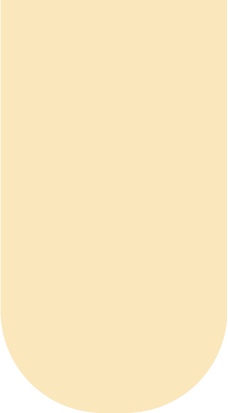
PHP supports several data types to classify information:

- **String** – sequence of characters ("Hello").
- **Integer** – whole numbers (25).
- **Float / Double** – numbers with decimals (3.14).
- **Boolean** – true or false values, often used in condition checks.
- **Array** – a collection of multiple values stored under one variable.

```
<?php
    $greeting = "Hello, World!";
    echo $greeting;
?>
```

KEY NOTES

- PHP runs entirely on the server-side, meaning all processing happens before sending the page to the browser.
- This makes it secure, as sensitive operations like authentication, database queries, or calculations are hidden from users.
- The syntax is simple, making it beginner-friendly while still powerful enough for complex applications.
- Combining PHP with HTML allows for dynamic and interactive web pages.



ROLE OF PHP AS A SERVER – SIDE LANGUAGE

“SERVER-SIDE”

Client-side languages (like HTML, CSS, JavaScript) run inside the user's browser. They control layout, style, and small interactions.

Server-side languages (like PHP) run on the web server before the content reaches the browser. They handle business logic, process data, interact with databases, and generate the actual page sent back to the client.

PHP'S EXECUTION PROCESS

1. A user fills out a form or clicks a link in their browser.
2. The browser sends a request to the web server.
3. If the request points to a .php file, the server hands it to the PHP interpreter.
4. PHP code is executed on the server: it may access databases, process form input, or perform calculations.
5. The output (usually HTML, sometimes JSON or XML) is sent back to the browser.
6. The browser displays the result, but the PHP code itself remains hidden.

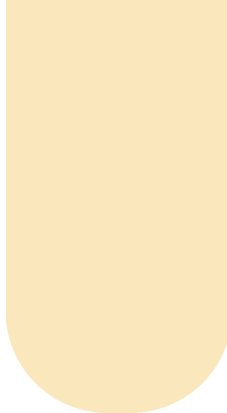
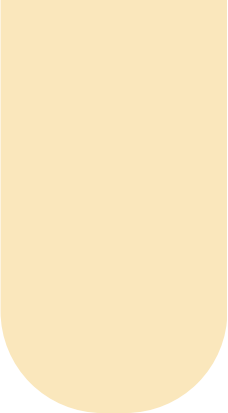
EXAMPLE IN ACTION

- HTML Form:

```
<form action="welcome.php" method="post">  
  Name: <input type="text" name="name">  
  <input type="submit">  
</form>
```

- welcome.php:

```
<?php  
  $name = $_POST['name'];  
  echo "Welcome, " . htmlspecialchars($name) . "!";  
?>
```



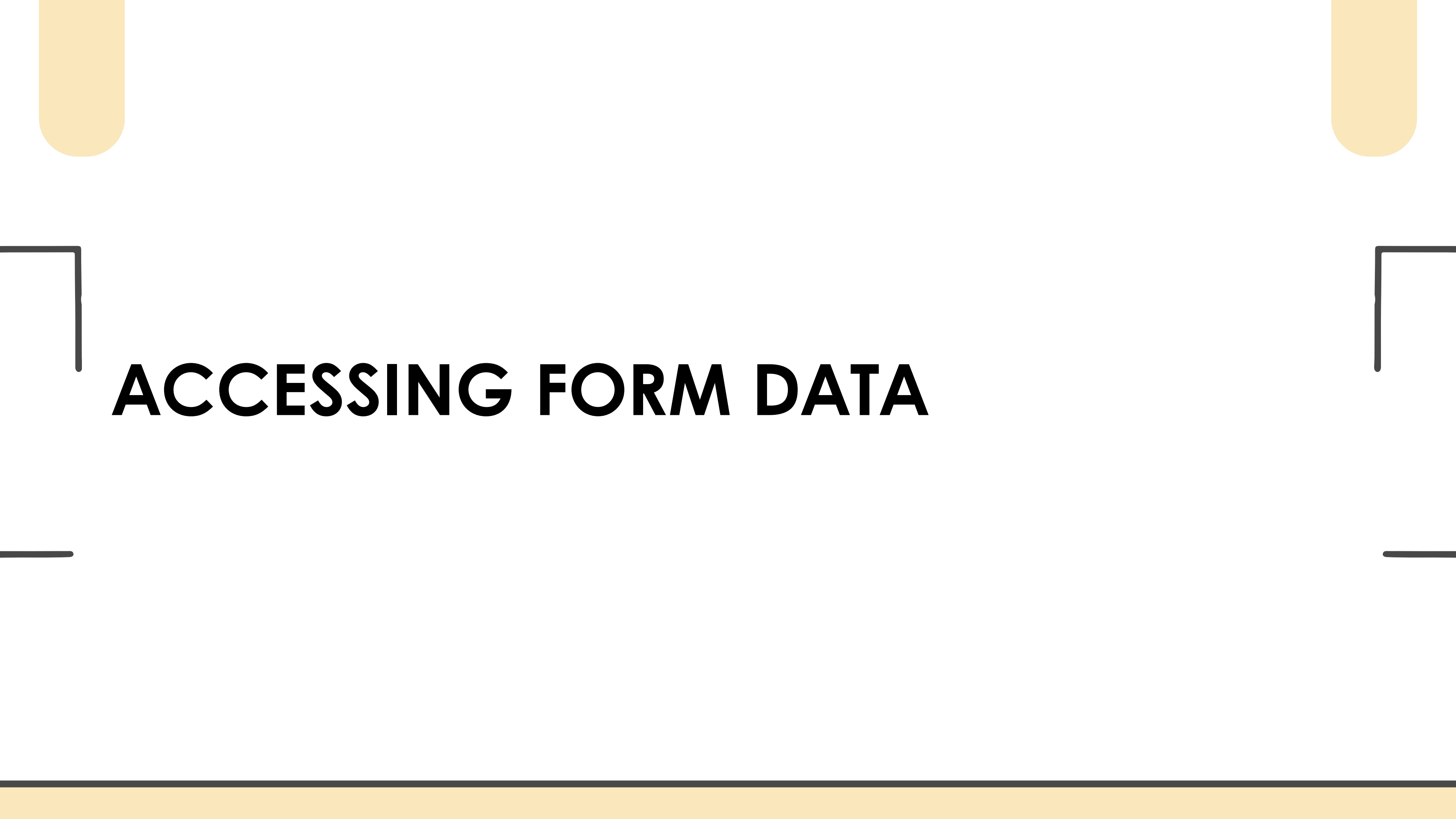
HANDLING CLIENT REQUESTS WITH PHP SUPERGLOBALS

WHAT ARE PHP SUPERGLOBALS?

In PHP, superglobals are special built-in variables that are always accessible from any part of a script, regardless of scope. They are predefined by PHP to allow developers to easily access data sent by the client (such as form input), information about the server, session data, cookies, or uploaded files. Superglobals are critical because they form the primary interface between the client (browser) and the server-side PHP script.

COMMON SUPERGLOBALS INCLUDE:

- ***\$_GET*** – retrieves data sent via the URL.
- ***\$_POST*** – retrieves data sent via the HTTP POST method.
- ***\$_REQUEST*** – contains combined GET, POST, and COOKIE data.
- ***\$_SERVER*** – contains server and execution environment information.
- ***\$_COOKIE*** – stores data saved in the client's browser.
- ***\$_SESSION*** – stores data across multiple pages on the server.
- ***\$_FILES*** – contains information about uploaded files.



ACCESSING FORM DATA

\$_GET

- The `$_GET` superglobal allows you to collect data sent through the URL query string.
- Example: A form sends data using `method="get"`:

```
http://example.com/page.php?name=Alex&age=13
```

- In `process.php`:

```
<?php  
echo "Hello " . $_GET["name"] . "! You are " . $_GET["age"] . " years  
old."  
?>
```

\$_POST

- The \$_POST superglobal collects data sent securely through HTTP POST, which does not display in the URL.
- Example: A login form:

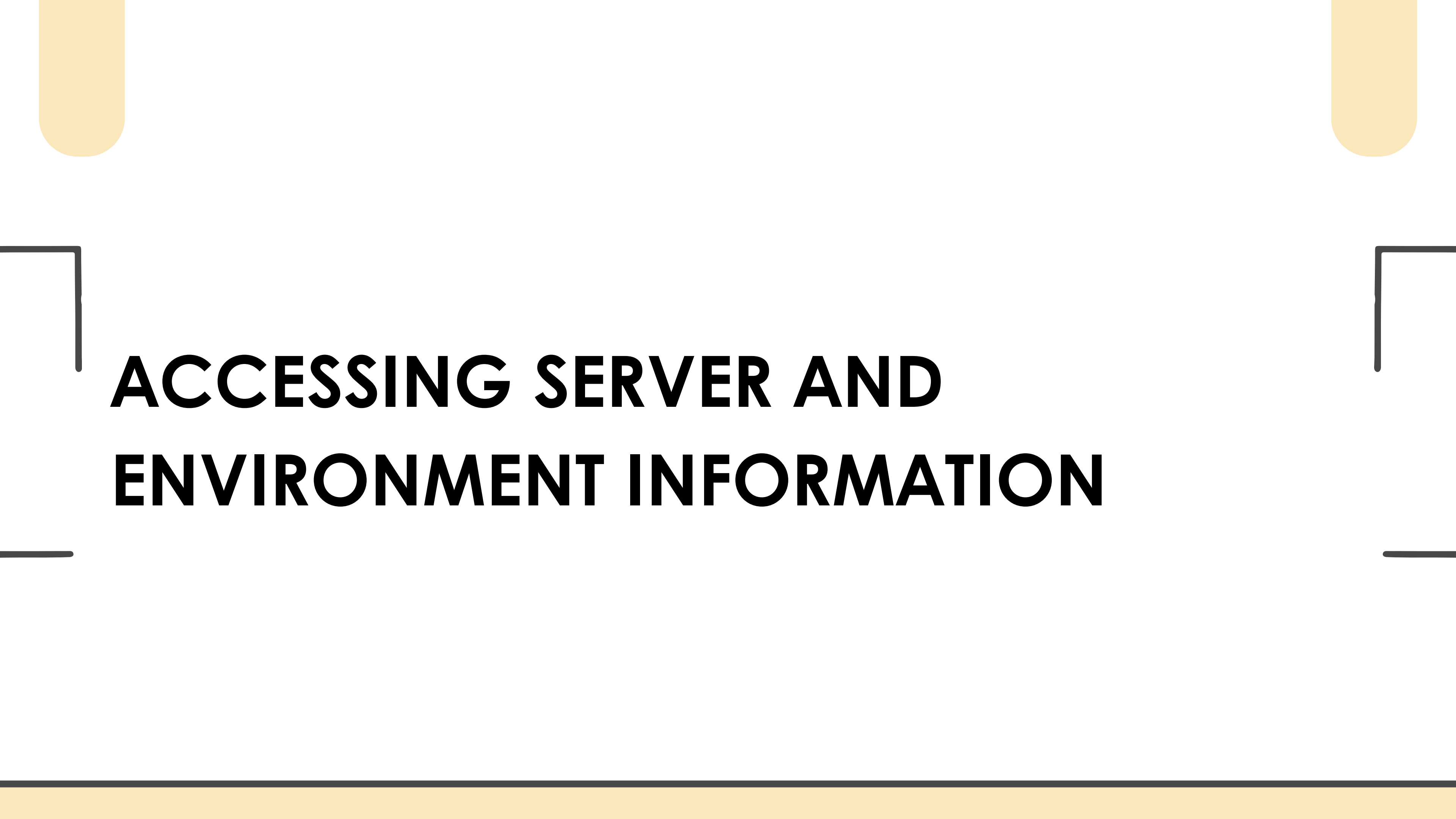
```
<form method="post" action="login.php">  
Username: <input type="text" name="user">  
Password: <input type="password" name="pass">  
<input type="submit">  
</form>
```

- In login.php:

```
$username = $_POST['user'];  
$password = $_POST['pass'];
```

\$_REQUEST

- **\$_REQUEST combines GET, POST, and COOKIE data in a single array.**
- **While convenient, it is less secure because it merges multiple sources.**



ACCESSING SERVER AND ENVIRONMENT INFORMATION

\$_SERVER

- This superglobal stores information about the server, client request, and execution environment.
- **Examples:**

```
echo $_SERVER['SERVER_NAME']; // Server hostname  
echo $_SERVER['REQUEST_METHOD']; // GET or POST  
echo $_SERVER['HTTP_USER_AGENT']; // Browser info
```


\$_COOKIE

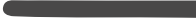
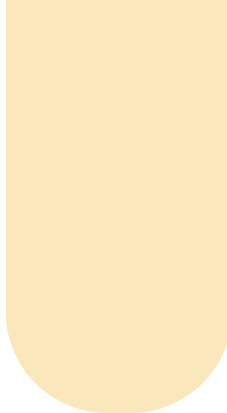
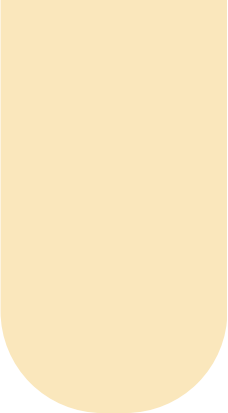
- Cookies are small pieces of data stored in the user's browser.
- PHP can set and retrieve cookies:

```
setcookie("user", "Alice", time()+3600);  
echo $_COOKIE['user'];
```

\$_SESSION

- Sessions store information on the server, allowing data to persist across multiple pages.
- Example:

```
session_start();  
$_SESSION['user'] = "Alice";  
echo $_SESSION['user'];
```



HANDLING FILE UPLOADS

\$_FILES

- This superglobal provides access to files uploaded from a client's browser.
- Example HTML form:

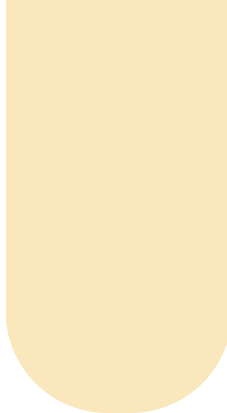
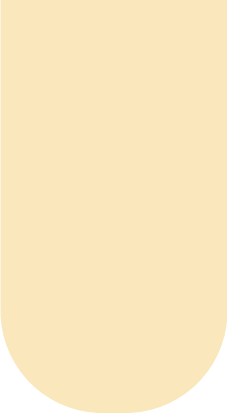
```
<form method="post" enctype="multipart/form-data"
action="upload.php">
<input type="file" name="profilePic">
<input type="submit">
</form>
```

- Accessing file data in PHP:

```
$filename = $_FILES['profilePic']['name'];
$tmpName = $_FILES['profilePic']['tmp_name'];
move_uploaded_file($tmpName, "uploads/".$filename);
```

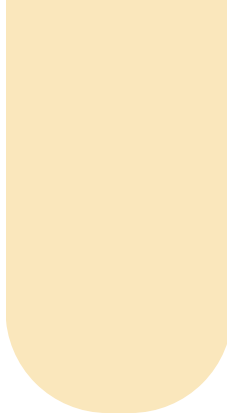
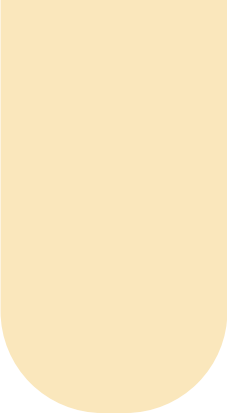
\$_FILES

- This superglobal provides access to files uploaded from a client's browser.
- Example HTML form:



PHP superglobals make it easy to interact with clients, collect input, manage sessions, and process files, forming the core of server-side programming in web applications.





CREATING AND DESIGNING HTML FORMS

DESIGNING FORMS FOR USABILITY

From Web Database Applications with PHP and MySQL and Programming PHP:

- *Keep forms simple and clear.*
- *Use labels for accessibility:*
- `<label for="username">Username:</label>`
- `<input type="text" id="username" name="username">`
- *Group related fields with <fieldset> and <legend>.*
- *Provide default values or placeholders to guide users.*
- *Always include a submit button.*

COMMON INPUT ELEMENTS

Forms can contain many input types:

- Text fields:

```
<input type="text" name="username">
```

- Password fields:

```
<input type="password" name="password">
```

- Radio buttons (choose one):

```
<input type="radio" name="gender" value="male"> Male  
<input type="radio" name="gender" value="female"> Female
```

COMMON INPUT ELEMENTS

Forms can contain many input types:

- Checkboxes (choose multiple):

```
<input type="checkbox" name="hobby" value="reading">  
Reading  
<input type="checkbox" name="hobby" value="sports"> Sports
```

- Dropdown menus:

```
<select name="country">  
  <option value="PH">Philippines</option>  
  <option value="US">United States</option>  
</select>
```

COMMON INPUT ELEMENTS

Forms can contain many input types:

- Text area (multi-line input):

```
<textarea name="message"></textarea>
```

- Submit button:

```
<input type="submit" value="Submit">
```


EXAMPLE: A BASIC LOGIN FORM

```
<form action="process.php" method="post">
<label for="username">Username:</label>
<input type="text" id="username" name="username" required><br><br>

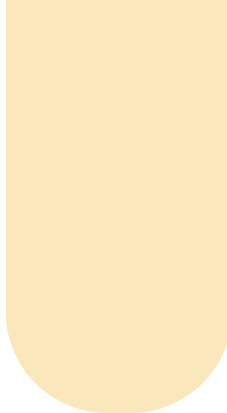
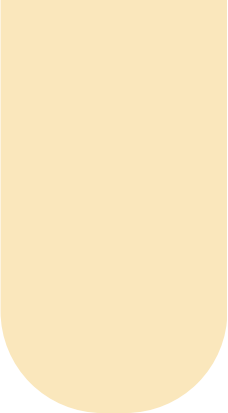
<label for="password">Password:</label>
<input type="password" id="password" name="password" required><br>
<br>

<input type="submit" value="Login">
</form>
```

- Key Features:
 - *required* → ensures the field is not left blank.
 - Labels make it user-friendly.
 - Data will be sent to *process.php* for handling.

WHY THIS MATTERS FOR PHP

- *PHP uses superglobals (\$_GET, \$_POST) to read data from these forms.*
- *Designing good forms is the first step in building interactive applications.*
- *Example: A registration form → PHP script stores data in a database → displays a welcome message.*



PROCESSING FORM DATA WITH PHP

HOW FORMS AND PHP WORK TOGETHER

When a user interacts with a website—for example, by entering their name, email, or password—they usually do so through HTML forms. These forms act like a digital questionnaire: they collect data and then send it to the web server for processing.

This is where PHP comes in. PHP acts as the “middleman” between the form (client-side) and the server. It receives the form data, processes it, validates it, and then either returns output to the browser or stores it in a database.

PHP FORM APPLICATION

1: Create the HTML Form (Client Side)

1. Open your code editor.
2. Create a new file and save it as profile_form.html
3. This form should ask for:
 - Full Name
 - Age
 - Favorite Hobby
 - Favorite Food
 - Gender (radio buttons)
 - Year Level (dropdown list)

PHP FORM APPLICATION

2: Create the PHP Script (Server Side)

1. Create another new file and save it as profile_process.php.
2. The script will:
 - Collect the form data using `$_POST`.
 - Display the data back as a mini-profile card.
 - Generate a dynamic message based on the hobby